

The question *worth* asking

Most systems that aren't sure who you meant either guess, or ask you about everything. We do a third thing: we work out which single question would tell us the most, and ask only that one.

Submission 7 Sep 2026 312 tests, all offline Benchmarked, not projected

THE SURFACE

You talk. It remembers people.

You've just left an event where you met eleven people. Two days later you remember four of them, and one of those only as "the girl from the robotics lab."

Recall is something you talk to. You record about ninety seconds of messy speech on the way home, and it turns that into contact records that build up over time — so the next memo, three weeks later, gets attached to the people you already know.

That part is the demo. Here is what a real memo sounds like:

"...and then there was Wei Lin, from GIC I think, she's hiring for some quant infrastructure role — said I'd send her the Kestrel repo. Oh and I ran into the malaysian chinese girl again, the one from the hall thing, she's doing the same module as me now..."

"Wei Lin, from GIC" is easy. "**The malaysian chinese girl**" is the interesting one — and it is the normal way people actually talk. That phrase is where this project lives.

THE PROBLEM

Two people on file both fit. Now what?

Every system that reads messy human text about people hits this exact fork. There are only two answers in common use, and both of them cost you something.

OPTION ONE

Guess, if you're confident enough

Score the match, and merge when the score clears a line. This is what almost everything does today.

What it costs: when it's wrong, two different people quietly become one record and nobody finds out until it matters. We hit this ourselves — two people merged because their names shared a single syllable.

OPTION TWO

Send everything uncertain to a human

Never guess. Put every unclear case in a queue and let a person sort it out.

What it costs: the queue never ends, it grows with your data, and the person gets no help deciding — just two records side by side. Eventually they stop reading.

There is a third option, and it is the whole project: keep the uncertainty, and spend one question on it.

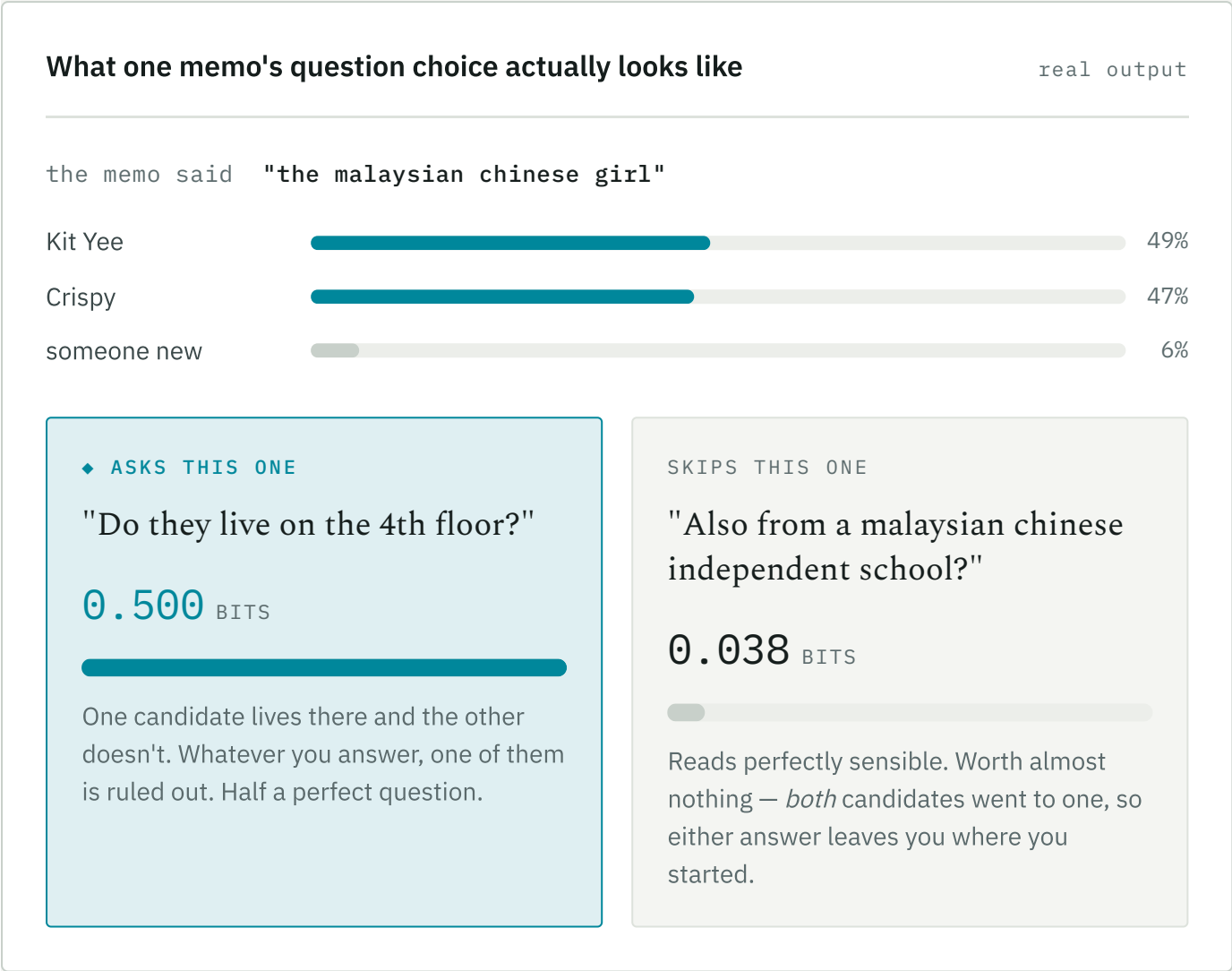
THE IDEA, IN PLAIN TERMS

It's twenty questions, played properly

Everyone has played twenty questions. The skill isn't guessing the answer — it's picking questions that cut the possibilities roughly in half. "Is it an animal?" is a good question. "Is it a badger?" usually isn't, because the answer is almost always no and you learn nearly nothing.

There is a way to measure that, and it has been around since 1948. The unit is the **bit**. One bit is exactly one perfect yes-or-no question — the kind that splits the possibilities down the middle. Half a bit is half as useful. Nought point nought three bits is a question barely worth the breath.

So before Recall asks you anything, it writes out every question it *could* ask, scores each one in bits, and asks the highest. Below is a real case, with the real numbers.



That second question is the point of the whole exhibit. It is exactly the question a language model produces when you ask it "what should I ask?", because it reads well and sounds relevant. It is worth **one thirteenth** of the question next to it.

The arithmetic knows. A prompt doesn't.

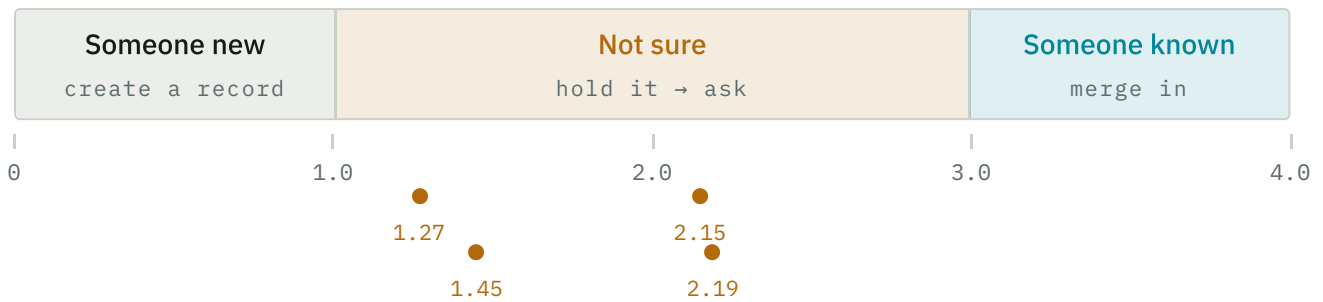
THE MECHANISM

Three answers, not two

Underneath, every mention gets a match score. Most systems draw one line on that score: above it, same person; below it, new person. We draw **two** lines, which creates a middle zone where the honest answer is "I don't know yet."

The match score, and what happens at each level

thresholds 1.0 / 3.0



The four dots are real scores from one test set — four mentions of people whose names all begin "Jia". Every one of them lands in the middle. A single-line system would have forced all four into a yes or a no; this one holds them and asks.

The part that took the longest to get right is not visible in that diagram. An earlier version *had already decided* before it asked — it worked out an answer, then asked you a question it was going to ignore. The question was theatre.

Now the system genuinely stops. The pipeline suspends mid-run, the uncertain person sits in neither pile, and **your answer is what decides which way they go**. That is a real pause, not a loading spinner.

THE RESULT

Does picking the question actually save you anything?

This is the number the project exists to produce. We ran the same recordings three ways: our method, and the two standard alternatives — asking about whatever the system is least sure about, and asking at random.

The measure is **questions per person correctly identified**. It counts how much of your attention it costs to get the right answer. **Lower is better**.

Questions asked per person resolved

3 full runs · lower is better

Ours — highest information gain 0.985 spread 0.839 – 1.061



Ask about the least certain 1.359 spread 1.242 – 1.429



Ask at random 1.323 spread 1.258 – 1.417



0 0.5 1.0 1.5

our method standard alternatives Whiskers show the best and worst of three runs.

THE SAME FIGURES AS DATA

STRATEGY	QUESTIONS EACH	BEST RUN	WORST RUN	DONE IN ≤1
Highest information gain	0.985	0.839	1.061	68%
Least certain first	1.359	1.242	1.429	57%
Random	1.323	1.258	1.417	60%

Roughly a third fewer interruptions for the same result. And the margin holds up under the test that matters: our *worst* run (1.061) was still better than the *best* run of either alternative (1.242 and 1.258). The ranges don't touch, so this isn't one lucky run.

Separately, on how well it recognises people at all — grouping the same person's mentions together across memos — it scores **0.927 out of 1.0** across five test sets. Nearly all of the remaining error is failing to recognise someone it should have, rather than merging two people who aren't the same. That is the right direction to fail in: a missed match is a small annoyance, a wrong merge is corrupted data.

HONESTY

What these numbers do not say

A result you can't poke holes in usually means nobody looked. Here is where ours is thin.

We beat both alternatives. We can't rank them against each other.

"Least certain" and "random" landed at 1.359 and 1.323, and their ranges overlap heavily. At three runs they are indistinguishable. Only our method separates from the pack.

The two headline numbers are not independent.

Recognition accuracy and question efficiency come from the same tuning. A setting that pushes more cases into the "not sure" zone gives the question benchmark more, and harder, cases to work with. They must be read together, not as two separate wins.

Almost everything was tested in one kind of setting.

Three of the five test sets come from a university hall or one orientation group, where everybody shares a course, a floor and an event. That shared context is exactly what makes two people look alike. Settings tuned here may not carry over to a work context where job titles and companies actively contradict each other.

One accuracy loss is the cost of not asking — which cuts our way.

When the benchmark runs unattended there is nobody to answer, so a fallback has to guess. On one test set that fallback made a wrong merge and dropped precision to 0.947. The uncertain zone had correctly flagged all four cases; the guesser is what got it wrong. It is a direct measure of what happens when you skip the question.

"This might be someone I've never met" is a placeholder.

The starting assumption for a brand-new person is a number we chose by judgement, not one we derived from data. It works, but we can't defend the specific value.

THE BUILD

What is actually running

312 tests, all offline — no credentials, no spend	53 hand-written memos across 5 test sets	108 references to 40 recurring people	0 AI calls in the question-scoring step
---	--	---	---

That last figure matters more than it looks. The decision about which question to ask is arithmetic — no model call, no prompt. It is cheap, it is the same every time, and you can

read the reason it made any choice. In a setting where somebody has to explain a decision to an auditor, that is not a nice-to-have.

Six steps run for every memo, in order:

- 01 **Transcribe** — speech to text
- 02 **Extract** — pull out who was mentioned, and drop the passing mentions
- 03 **Compare** — score each mention against everyone already known
- 04 **Ask** — if the score lands in the middle, pick one question and stop for an answer
- 05 **Place** — file them as new, or merge into the record they belong to
- 06 **Summarise** — write it all back and report what changed

BEYOND THE DEMO

Networking is where we showed it, not where it ends

We chose event memos because they produce genuinely confusing references cheaply, and because you can watch the whole thing work in five minutes. The machinery underneath doesn't care that the entities are people you met at a party.

The same three parts — hold the uncertainty, work out which question would settle it, ask only that one — apply anywhere messy human text has to be matched against records that already exist. Customer databases full of duplicates. Job applicants who applied three times under two spellings of their name. Compliance analysts clearing name-match alerts by hand.

In all of those, the thing being spent is somebody's attention, and it has a price per minute. Fewer questions per resolution is not an academic score. It is a cost line.